Taller 8

Nicole Camila Abril

Gabriel Enrique Pacheco

Santiago Peña Pacheco

Julieta Quevedo González

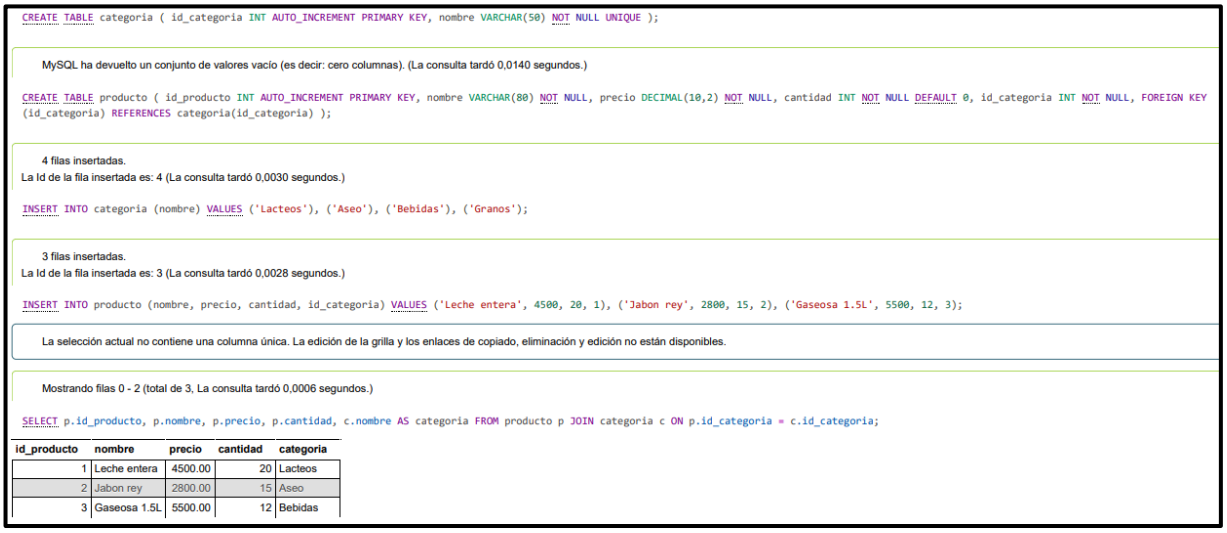
Programación I

Docente: Andrés Castaño

Fecha de entrega: 19/10/2026

Ejercicio 1: Base de datos.

Ingresando el código encontrado en el documento se creó la base de datos con sus tres productos iniciales, como se ve en la imagen:



```
CREATE TABLE categoria ( id_categoria INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(50) NOT NULL UNIQUE );
```

MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0140 segundos.)

```
CREATE TABLE producto ( id_producto INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(80) NOT NULL, precio DECIMAL(10,2) NOT NULL, cantidad INT NOT NULL DEFAULT 0, id_categoria INT NOT NULL, FOREIGN KEY
(id_categoria) REFERENCES categoria(id_categoria) );
```

4 filas insertadas.
La Id de la fila insertada es: 4 (La consulta tardó 0,0030 segundos.)

```
INSERT INTO categoria (nombre) VALUES ('Lacteos'), ('Aseo'), ('Bebidas'), ('Granos');
```

3 filas insertadas.
La Id de la fila insertada es: 3 (La consulta tardó 0,0028 segundos.)

```
INSERT INTO producto (nombre, precio, cantidad, id_categoria) VALUES ('Leche entera', 4500, 20, 1), ('Jabon rey', 2800, 15, 2), ('Gaseosa 1.5L', 5500, 12, 3);
```

La selección actual no contiene una columna única. La edición de la grilla y los enlaces de copiado, eliminación y edición no están disponibles.

Mostrando filas 0 - 2 (total de 3, La consulta tardó 0,0006 segundos.)

```
SELECT p.id_producto, p.nombre, p.precio, p.cantidad, c.nombre AS categoria FROM producto p JOIN categoria c ON p.id_categoria = c.id_categoria;
```

id_producto	nombre	precio	cantidad	categoria
1	Leche entera	4500.00	20	Lacteos
2	Jabon rey	2800.00	15	Aseo
3	Gaseosa 1.5L	5500.00	12	Bebidas

Figura 1: Creación de la base de datos

Ejercicio 2: Proyecto Maven y conexión

Tras realizar la clase que almacena la conexión con MySQL, se agregó la excepción que captura los errores en case de una conexión fallida con la base de datos.

En caso de error se muestra el siguiente mensaje:

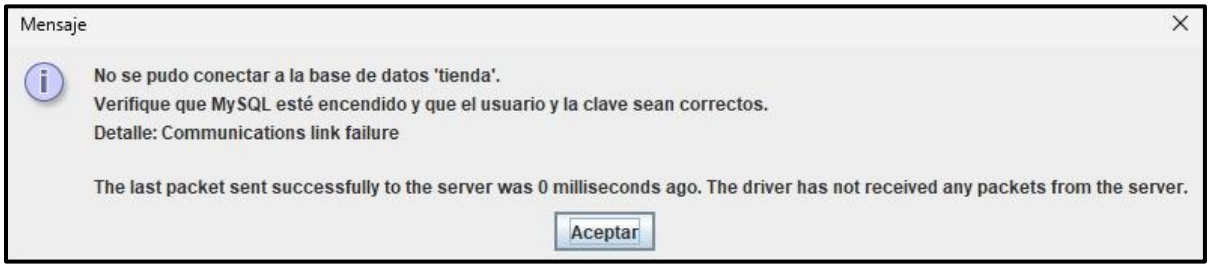
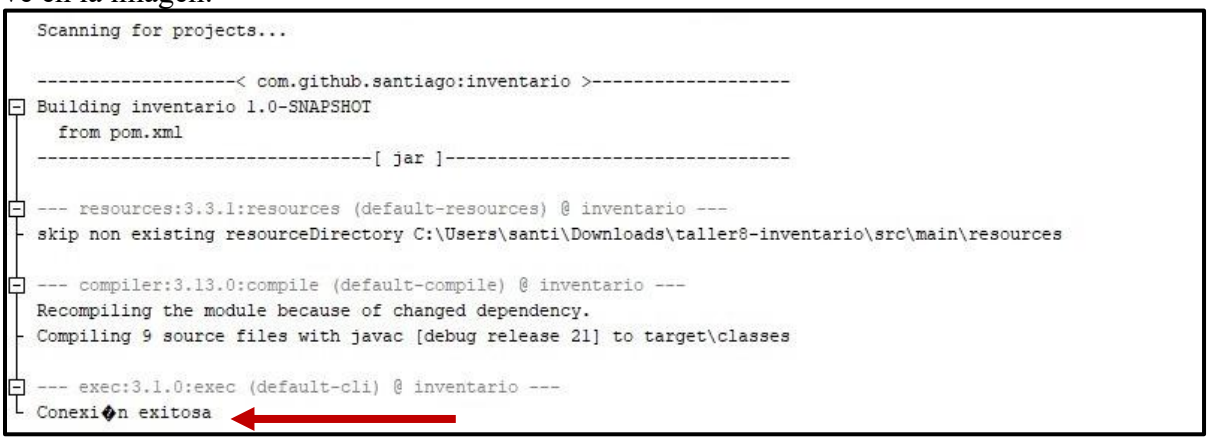


Figura 2: Mensaje de error al estar la base de datos apagada

En caso de no encontrar error se muestra en la consola el mensaje “Conexión exitosa” como se ve en la imagen:



```
Scanning for projects...

-------------------< com.github.santiago:inventario >--------------------
Building inventario 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.3.1:resources (default-resources) @ inventario ---
skip non existing resourceDirectory C:\Users\santi\Downloads\taller8-inventario\src\main\resources

--- compiler:3.13.0:compile (default-compile) @ inventario ---
Recompiling the module because of changed dependency.
Compiling 9 source files with javac [debug release 21] to target\classes

--- exec:3.1.0:exec (default-cli) @ inventario ---
Conexi◆n exitosa
```

Figura 3: Mensaje de conexión éxitos

Ejercicio 3 y 4: Clases de modelo y Capa DAO

En el repositorio de GitHub se encuentran las carpetas que contienen las diversas clases creadas para la visualización, tanto para las categorías y los productos como para el DAO y que se comunique con la base de datos.

Ejercicio 5: Interfaz gráfica

Se realizo la interfaz gráfica tomando como referencia el diseño propuesto en el documento, incluyendo las funciones de guardar, actualizar, eliminar, y limpiar, así como los campos para agregar los datos de cada producto. A continuación, una imagen de la interfaz realizada:

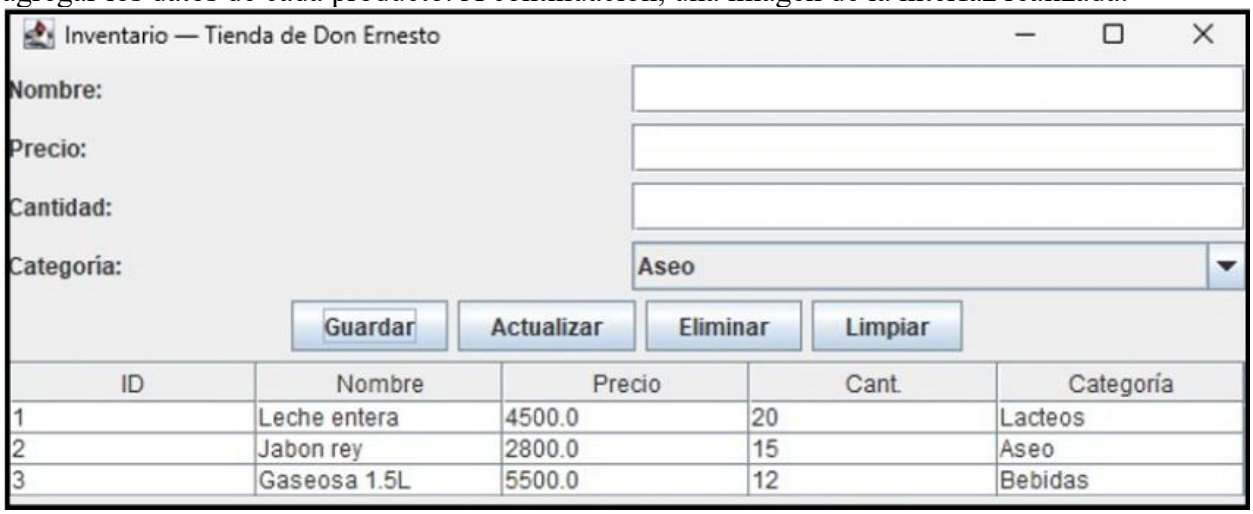


Figura 4: Interfaz grafica

Ejercicio 5: Prueba de persistencia y reflexión

A.1) Registro de productos

Para registrar un producto, se diligenciaron todas sus casillas con la información correspondiente y seleccionamos la opción de “Guardar” para confirmar su registro. Como se ve en las siguientes imágenes:

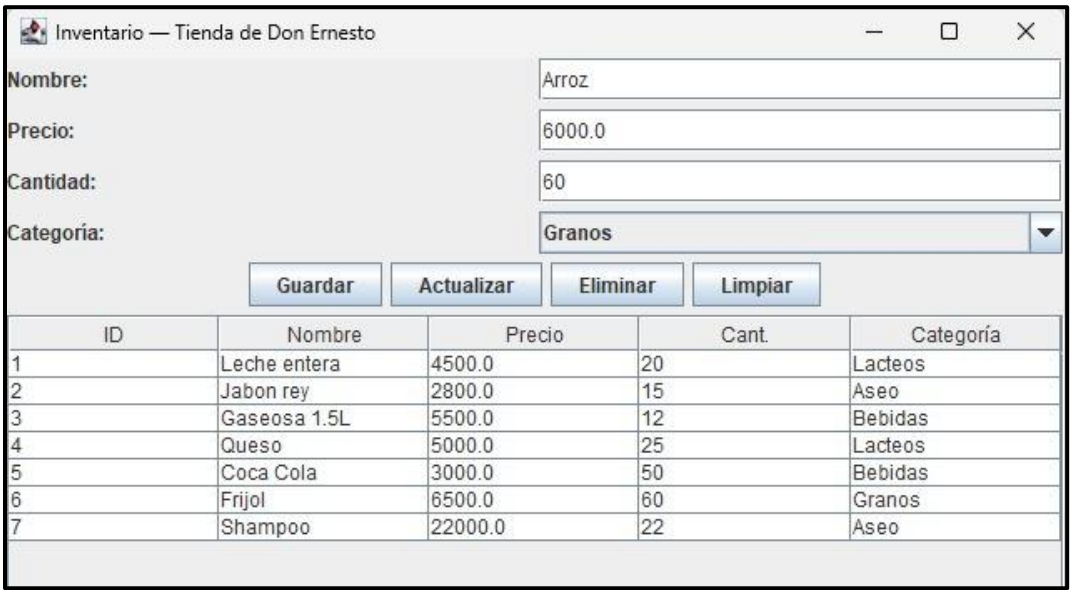


Figura 5: Interfaz agregando el producto "arroz"

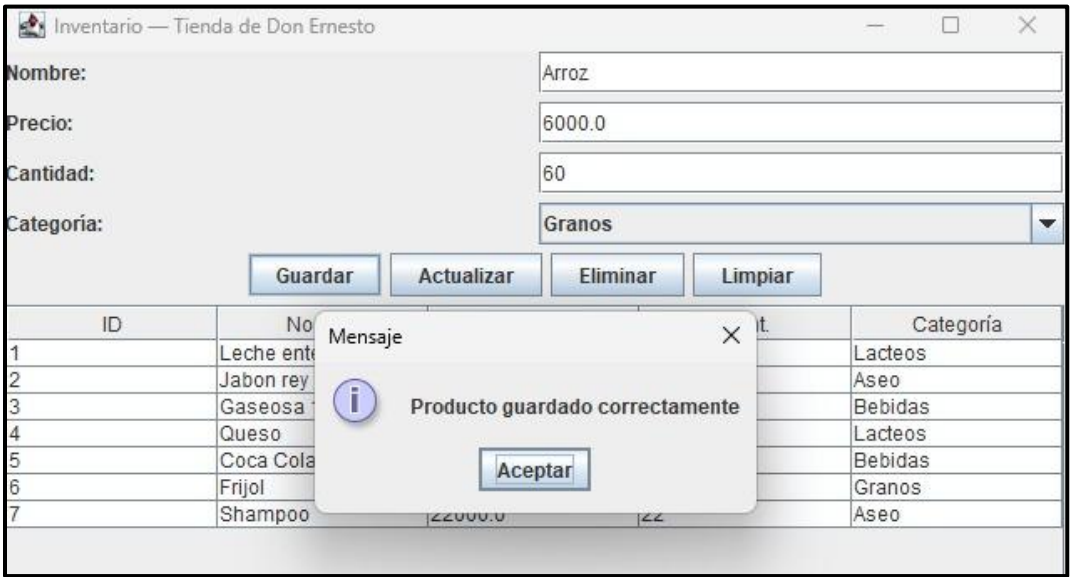


Figura 6: Mensaje de confirmación al registra el producto

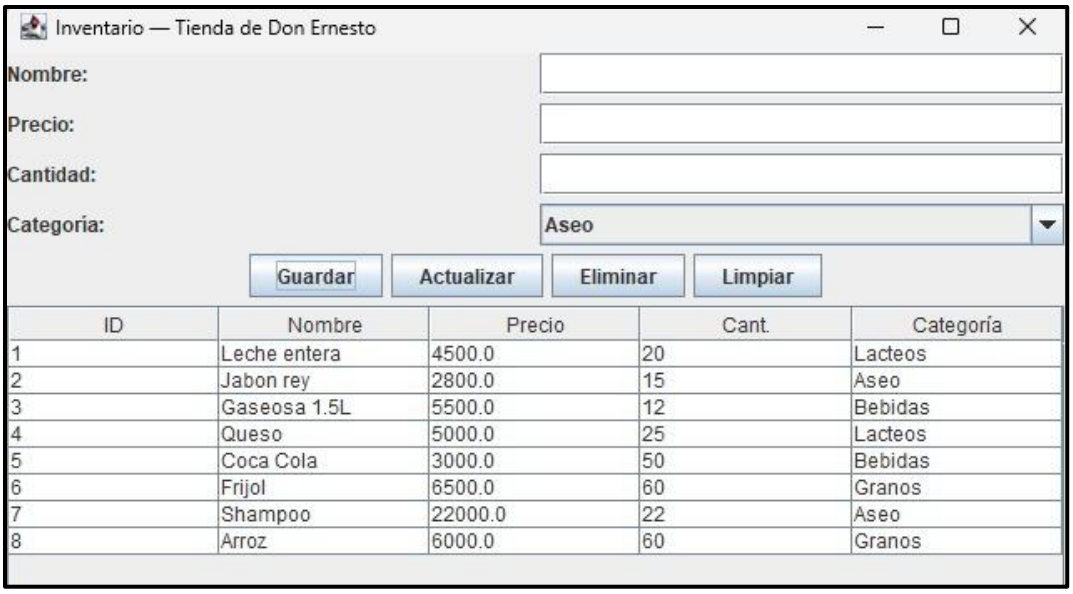


Figura 7: Tabla actualizada con todos los productos nuevos

A.2) Actualización de un producto

Para actualizar un producto, primero lo seleccionamos de la lista. Al tener su información dentro de los espacios editables, seleccionamos el dato a cambiar, lo editamos y confirmamos en "Actualizar". Como se ve en las siguientes imágenes:

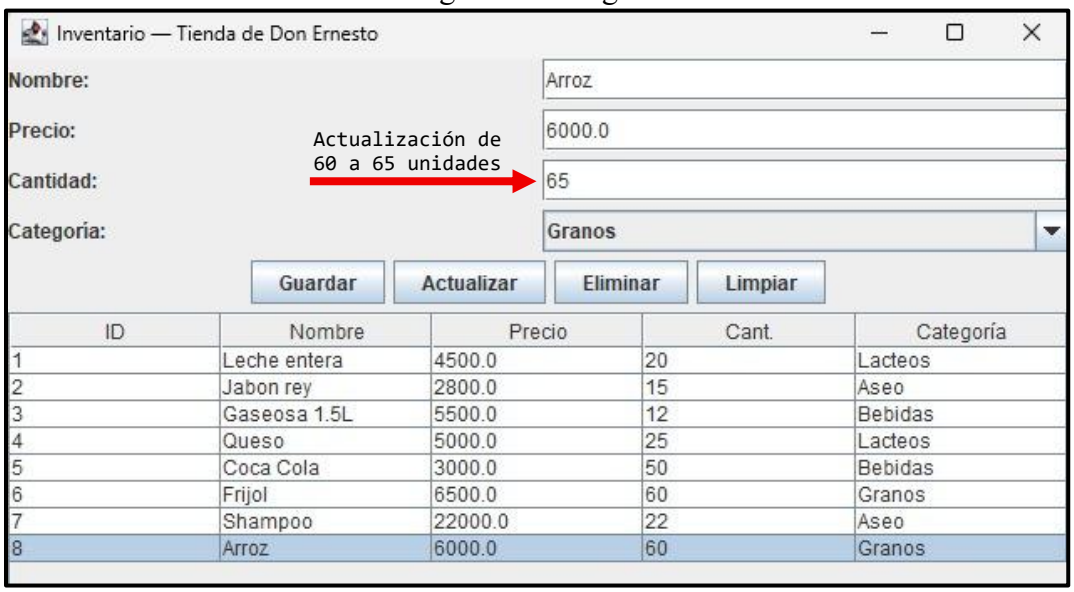


Figura 8: Selección de un producto ya existente (Arroz)



Figura 9: Mensaje de confirmación al actualizar el producto

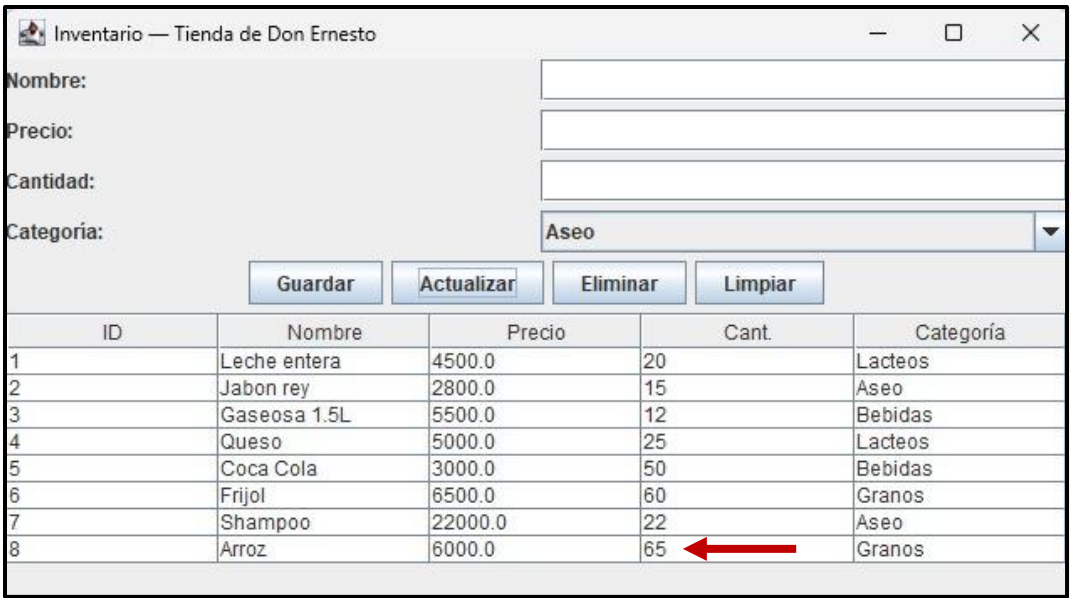


Figura 10: Tabla con datos actualizados

A.3) Eliminación de un producto

Para eliminar un producto, primero lo seleccionamos de la lista y seleccionamos la opción de "Eliminar". Como se ve en las siguientes imágenes:

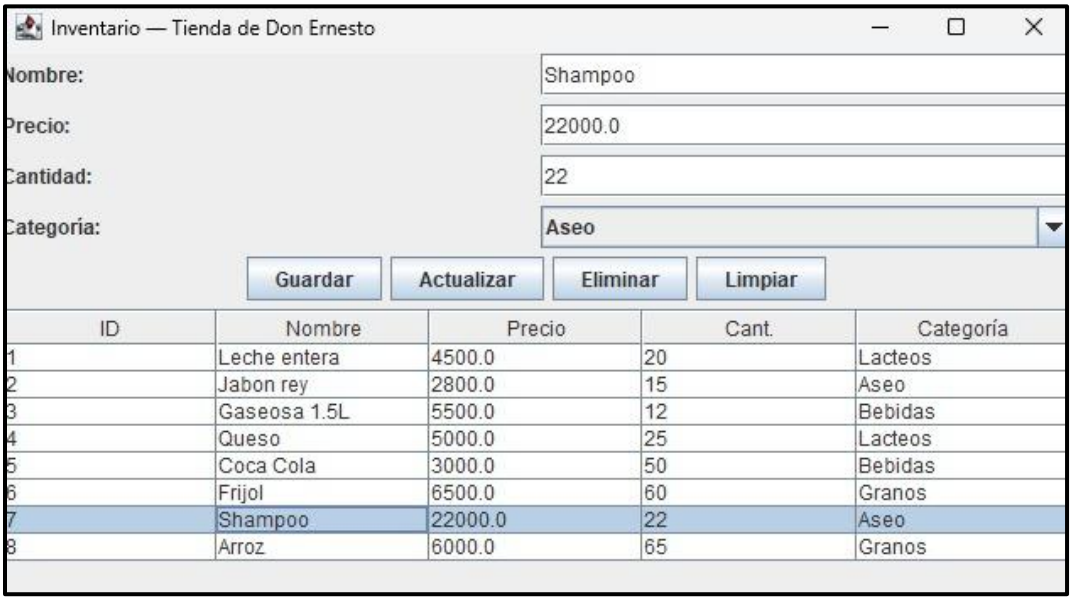


Figura 11: Selección de un producto ya existente (Shampoo)

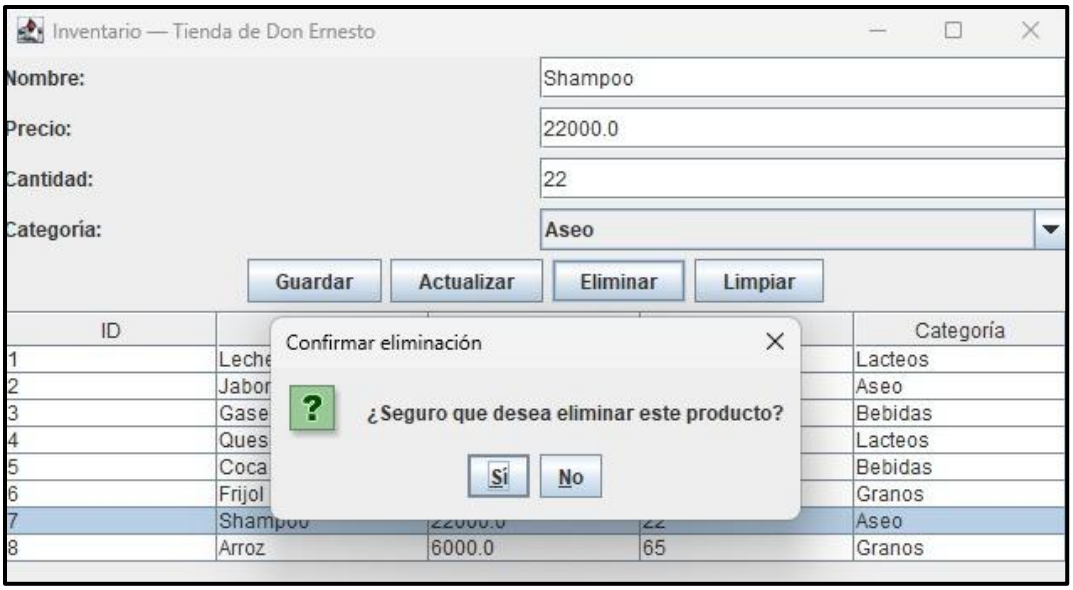


Figura 12: Mensaje de confirmación al eliminar el producto

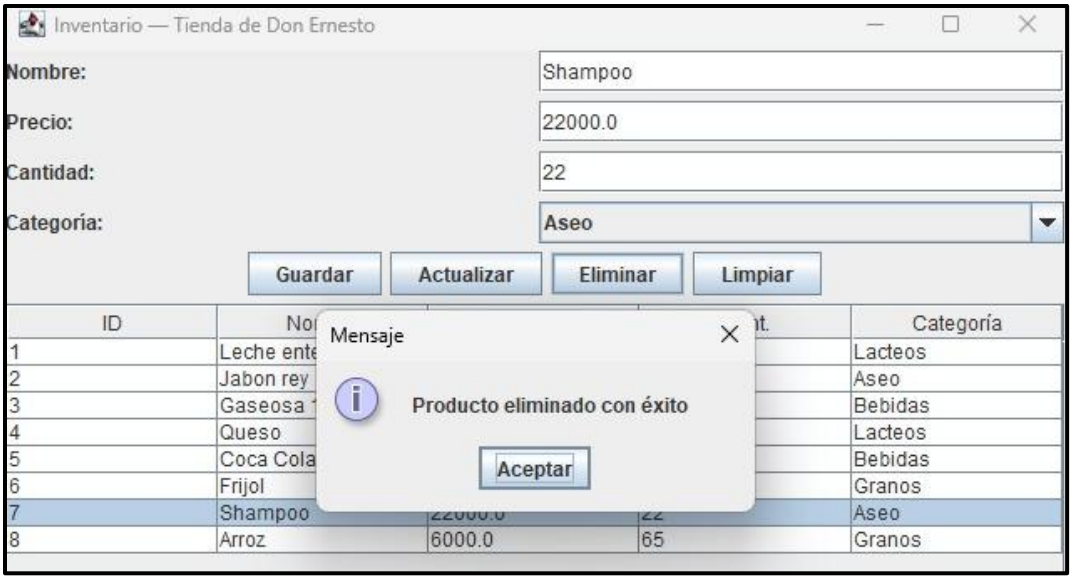


Figura 13: Confirmación de eliminación del producto



ID	Nombre	Precio	Cant.	Categoría
1	Leche entera	4500.0	20	Lacteos
2	Jabon rey	2800.0	15	Aseo
3	Gaseosa 1.5L	5500.0	12	Bebidas
4	Queso	5000.0	25	Lacteos
5	Coca Cola	3000.0	50	Bebidas
6	Frijol	6500.0	60	Granos
8	Arroz	6000.0	65	Granos

Figura 14: Tabla con datos actualizados (sin Shampoo)

A.4) Cierre de aplicación

Tras cerrar la aplicación y volver a ejecutarla, se mantienen los datos gracias al DAO, que permite visualizar los datos de la base de datos y actualizar la tabla de la aplicación con los datos existentes.



ID	Nombre	Precio	Cant.	Categoría
1	Leche entera	4500.0	20	Lacteos
2	Jabon rey	2800.0	15	Aseo
3	Gaseosa 1.5L	5500.0	12	Bebidas
4	Queso	5000.0	25	Lacteos
5	Coca Cola	3000.0	50	Bebidas
6	Frijol	6500.0	60	Granos
8	Arroz	6000.0	65	Granos

Figura 15: Tabla tras cerrar la aplicación

B) Verifique en MySQL

La verificación se realizó con el comando:
SELECT p.id_producto, p.nombre, p.precio, p.cantidad, c.nombre AS categoria
FROM producto p JOIN categoria c ON p.id_categoria = c.id_categoria

Que permitió ver los productos en cada categoría y se comparó con la aplicación como se ve en la imagen:

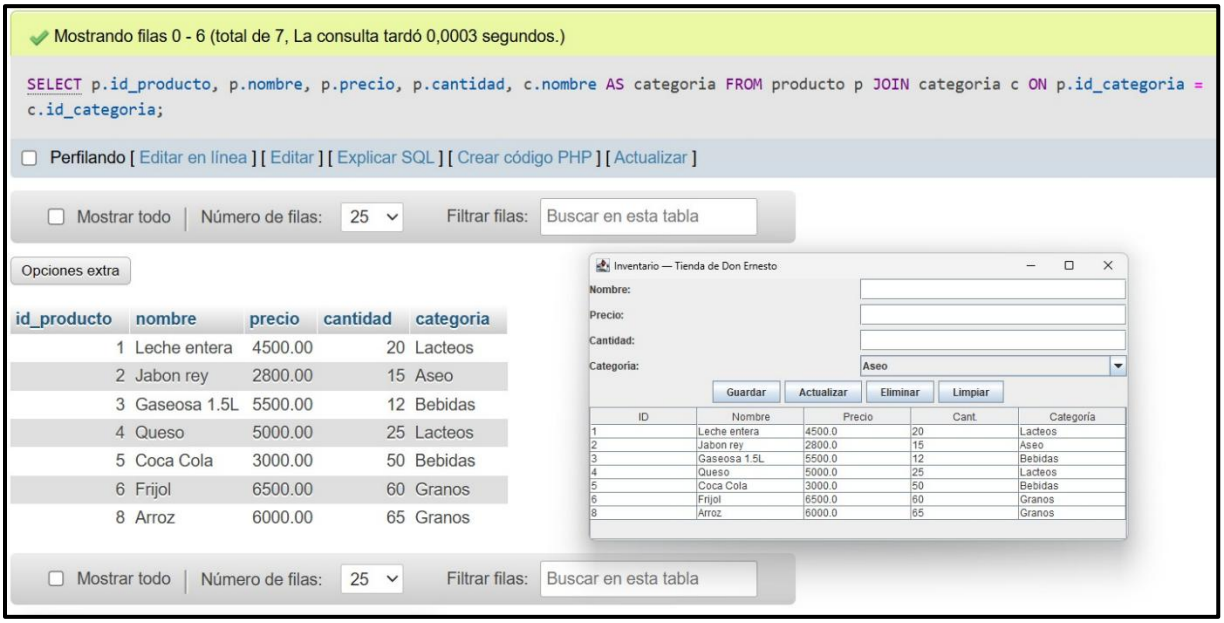


Figura 16: MySQL comparado con la tabla de la aplicación

C.1) Eliminación desde MySQL

Tras intentar eliminar la tercera categoría en la base de datos por medio de la consola con el script:
DELETE FROM categoria
WHERE id_categoria = 3;

Sin embargo, la operación produjo un error debido es justamente una llave foránea de sus productos asociados, por tal motivo no se pudo realizar su eliminación. A continuación, una imagen del error:

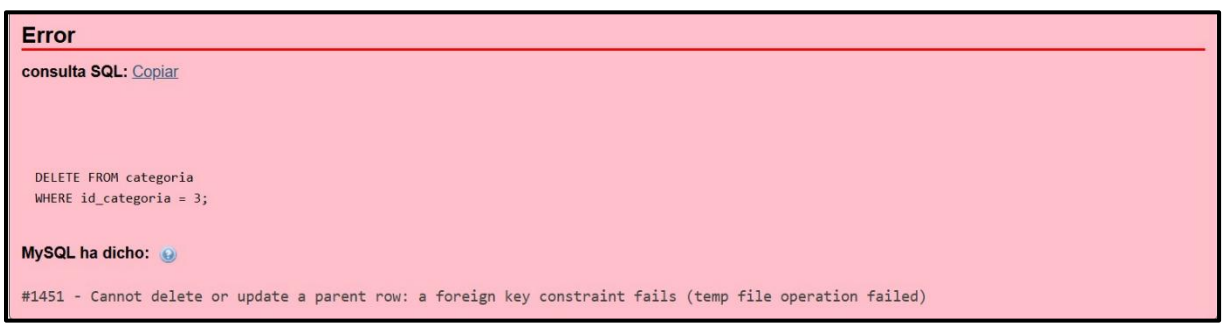


Figura 17: Error al intentar realizar la eliminación

C.2) ¿Qué ocurre y por qué?

MySQL rechaza la operación y arroja un error de restricción para evitar que productos queden sin categoría y, por tal motivo sin dicha ID. Esta acción evita inconsistencias y una posible pérdida de datos al no poder acceder a ellos debido a la restricción de la llave foránea o al no poder asociarlos justamente por falta de esta llave. Por tal razón, MySQL evita el proceso y mantiene la integridad de los datos.

C.3) ¿Cómo se relaciona esto con la composición entre Producto y Categoría?

La relación entre Producto y Categoría representa una estructura de tipo asociación o dependencia, ya que una Categoría agrupa a muchos Productos, pero un Producto pertenece a una única Categoría. Esto genera que producto posea la llave foránea de una categoría y, por ende, se referencie a esta. Por tal motivo, al eliminar una categoría, se tendría que eliminar o dejar de referenciar en los productos, lo cual no se puede realizar ya que podría suponer una pérdida de los propios productos.

D.1) ¿Qué ventaja tiene haber separado el proyecto en los paquetes conexión, model, dao y vista?

La principal ventaja es separar las funciones en cada clase para que se puedan encontrar comandos o acciones de forma sencilla. Esto sigue el primer principio SOLID (single responsbility), de responsabilidad única, permitiendo editar el código con facilidad y evitar un colapso total al mantener funciones separadas e impedir una gran cantidad de dependencias o composición entre clases.

D.2) ¿Qué tendría que cambiar si mañana la tienda decide usar PostgreSQL en lugar de MySQL?

Si la tienda pasara a PostgreSQL, habría que cambiar la dependencia del archivo pom.xml y la URL en Conexión, además revisar el DAO para actualizarlo al nuevo modelo de PostgreSQL. Esto ya que el pom.xml permite la comunicación, al igual que la clase “conexión” en general. Mientras el DAO, al ser el que comunica los datos entre la aplicación y la base de datos, también tendría que cambiar para localizar la nueva base de datos PostgreSQL.